NextSPICE
電路模擬引擎技術說明書

版本 0.5 — Subcircuit Supported Edition

March 26, 2026

涵蓋系統架構、編譯器原理、數學推導、
模擬演算法與完整程式碼流程分析

Contents

1 系統總覽 **2**
- 1.1 NextSPICE 簡介 . . . 2
- 1.2 系統架構分層 . . . 2
- 1.3 原始碼檔案總覽 . . . 2

2 網表語法與編譯器 **3**
- 2.1 SPICE 網表格式 . . . 3
  - 2.1.1 元件語法 . . . 3
  - 2.1.2 分析指令語法 . . . 4
- 2.2 單位轉換系統 . . . 4
- 2.3 編譯流程 . . . 4
- 2.4 子電路展平演算法 . . . 5

3 修正節點分析法（MNA） **6**
- 3.1 理論基礎 . . . 6
- 3.2 元件戳印法（Stamping） . . . 6
  - 3.2.1 電阻器 . . . 6
  - 3.2.2 電壓源 . . . 7
  - 3.2.3 電流源 . . . 7
  - 3.2.4 電容器 . . . 7
  - 3.2.5 電感器 . . . 7
  - 3.2.6 互感（Mutual Inductance） . . . 8
  - 3.2.7 電壓控制電壓源（VCVS，E 元件） . . . 8
  - 3.2.8 電壓控制電流源（VCCS，G 元件） . . . 8
  - 3.2.9 電流控制電壓源（CCVS，H 元件） . . . 9
  - 3.2.10 電流控制電流源（CCCS，F 元件） . . . 9
- 3.3 節點管理器 . . . 9

4 時域波形產生器 **9**
- 4.1 正弦波 SIN . . . 9
- 4.2 脈衝波 PULSE . . . 10
- 4.3 分段線性波 PWL . . . 10

5 分析演算法 **10**
- 5.1 直流工作點分析（.OP） . . . 10
- 5.2 交流小訊號分析（.AC） . . . 11
- 5.3 直流掃描分析（.DC） . . . 11
- 5.4 暫態分析（.TRAN） . . . 11
  - 5.4.1 後向歐拉法原理 . . . 12
  - 5.4.2 伴隨電路模型（Companion Model） . . . 12
  - 5.4.3 暫態分析流程 . . . 12
- 5.5 靈敏度分析（.SENS） . . . 12
  - 5.5.1 直流增益定義 . . . 12
  - 5.5.2 中央差分靈敏度 . . . 13
- 5.6 參數掃描（.STEP） . . . 13

6 量測引擎 **13**
6.1 .MEASURE TRAN 指令 13
6.1.1 TRIG...TARG 雙點測量語法 13
6.1.2 穿越點偵測演算法 13

7 稀疏矩陣求解策略 **14**
7.1 為何使用稀疏矩陣 14
7.2 SciPy 稀疏矩陣工作流程 14

8 電路建構器（Two-Pass 建構法） **14**
8.1 Pass 1：獨立元件建構 14
8.2 Pass 2：交叉參照解析 15

9 分析排程總管 **15**
9.1 結果輸出格式 15
9.2 功率計算 16

10 支援的元件總覽 **16**

11 完整模擬範例 **16**
11.1 範例一：RC 低通濾波器頻率響應 16
11.2 範例二：RLC 暫態響應 17
11.3 範例三：變壓器互感模型 17
11.4 範例四：含參數的靈敏度分析 17

12 端到端資料流程圖 **18**

13 已知限制與未來展望 **19**
13.1 目前限制 19
13.2 效能特性 19

附錄 A：數學符號表 **19**

附錄 B：SPICE 網表快速參考 **20**

第 1 章　系統總覽

1.1　NextSPICE 簡介

NextSPICE 是一套以 Python 撰寫的開源類比電路模擬引擎，實作了業界標準 SPICE（Simulation Program with Integrated Circuit Emphasis）的核心功能子集。系統支援以下五種分析模式：

(1) **直流工作點分析**（`.OP`）— 求解電路的穩態直流解。

(2) **交流小訊號分析**（`.AC`）— 在頻域中掃描阻抗響應。

(3) **直流掃描分析**（`.DC`）— 對某一獨立源進行參數掃描。

(4) **暫態分析**（`.TRAN`）— 以時域數值積分模擬動態行為。

(5) **靈敏度分析**（`.SENS`）— 以中央差分法求各元件對增益的靈敏度。

1.2　系統架構分層

NextSPICE 採用四層式架構，各層職責分明：

層級	名稱	職責
1	前端層（Frontend）	FastAPI Web 伺服器，提供 REST API
2	編譯層（Compiler）	網表解析、參數展開、驗證、子電路展平
3	引擎層（Engine）	元件模型定義、矩陣代數運算
4	執行層（Runtime）	電路建構、求解器、分析排程、量測引擎

資料流向為：

網表文字 $\xrightarrow{\text{Compiler}}$ JSON 電路描述 $\xrightarrow{\text{Runtime Builder}}$ Python 元件物件 $\xrightarrow{\text{Solver}}$ 模擬結果

1.3　原始碼檔案總覽

目錄	檔案	功能說明
`compiler/`	`frontend.py`	編譯器主協調器，子電路展平
	`preprocess.py`	預處理：註解過濾、行延續、分詞
	`parse_elements.py`	元件語法解析 (R, L, C, V, I, E, G, H, F, K, D, X)
	`parse_directives.py`	分析指令解析 (.OP, .AC, .TRAN 等)
	`param_eval.py`	`.PARAM` 參數運算環境
	`validator.py`	電路驗證（接地檢查、浮接偵測）
	`formatter.py`	網表美化排版
	`diagnostics.py`	錯誤報告基礎設施
`engine/`	`elements.py`	所有元件的 MNA stamp 實作
	`matrix.py`	抽象矩陣基底類別與稠密矩陣實作
`runtime/`	`circuit.py`	電路建構器（Two-Pass 建構法）
	`solver.py`	稀疏矩陣求解器（v0.4 SciPy Sparse）

目錄	檔案	功能說明
	runner.py measure.py	分析排程總管（含.STEP 掃描） .MEASURE 量測引擎
utils/	unit_conv.py	SPICE 單位轉換器
frontend/	app.py	FastAPI Web API 端點
根目錄	main.py	CLI 批次模擬入口點

第 2 章　網表語法與編譯器

2.1　SPICE 網表格式

NextSPICE 接受標準 SPICE 格式的純文字網表。基本語法規則如下：

- 第一行為電路標題（Title Line）。
- 以 * 開頭的行為註解。
- 以 + 開頭的行為前一行的延續。
- ; 或 $ 後方的文字為行內註解。
- .END 標記網表結束。
- 節點名稱 0、GND、GROUND 均代表接地。

2.1.1　元件語法

```
* 被動元件
R1 IN OUT 1k            ; 電阻 1k歐姆
C1 OUT 0 100n           ; 電容 100nF
L1 IN N1 10m            ; 電感 10mH
K1 L1 L2 0.99           ; 互感，耦合係數 0.99

* 獨立源
V1 IN 0 DC 10                           ; 直流電壓源
V2 IN 0 AC 1 0                          ; 交流源（振幅 1V, 相位 0°)
V3 IN 0 SIN(0 5 1k 0 0)                ; 正弦波
V4 IN 0 PULSE(0 5 1m 1u 1u 4m 10m)     ; 脈衝波
V5 IN 0 PWL(0 0 1m 5 2m 5 3m 0)        ; 分段線性波

* 受控源
E1 OUT 0 IN 0 100       ; VCVS, 增益 100
G1 OUT 0 IN 0 0.1       ; VCCS, 轉導 0.1 S
H1 OUT 0 V1 50          ; CCVS, 轉阻 50 Ω
F1 OUT 0 V1 2           ; CCCS, 電流增益 2

* 子電路呼叫
X1 IN OUT VCC 0 OPAMP   ; 呼叫 OPAMP 子電路
```

Listing 1: 元件定義範例

2.1.2 分析指令語法

```
.OP                                  ; 直流工作點
.AC DEC 100 10 1MEG                  ; 交流掃描: 對數, 100點/十倍頻, 10
   Hz~1MHz
.DC V1 0 10 0.1                      ; 直流掃描: V1 從 0V 到 10V, 步進
   0.1V
.TRAN 10u 10m                        ; 暫態分析: 步長 10 s, 終止 10ms
.SENS V(OUT) V1                      ; 靈敏度: 輸出 V(OUT), 輸入 V1
.STEP PARAM R_VAL 1k 10k 1k        ; 參數掃描
.PARAM R_VAL=1k                      ; 參數定義
.MEASURE TRAN delay TRIG V(IN) VAL=2.5 RISE=1
+                   TARG V(OUT) VAL=2.5 RISE=1
```

Listing 2: 分析指令範例

2.2 單位轉換系統

NextSPICE 實作了嚴格的 SPICE 單位轉換器（`unit_conv.py`），支援以下乘數後綴：

後綴	名稱	倍率	範例
T	Tera	10^{12}	1T $= 10^{12}$
G	Giga	10^{9}	2.2G $= 2.2 \times 10^{9}$
MEG	Mega	10^{6}	1MEG $= 10^{6}$
K	Kilo	10^{3}	4.7K $= 4700$
M	Milli	10^{-3}	10M $= 0.01$
U	Micro	10^{-6}	100U $= 10^{-4}$
N	Nano	10^{-9}	47N $= 4.7 \times 10^{-8}$
P	Pico	10^{-12}	22P $= 2.2 \times 10^{-11}$
F	Femto	10^{-15}	1F $= 10^{-15}$
MIL	Mil	25.4×10^{-6}	印刷電路板單位

乘數後方可接合法物理單位（V, A, OHM, F, H, S 等），例如 1kOHM、10uF。系統以正則表達式嚴格驗證格式，任何非法後綴都會拋出錯誤。

2.3 編譯流程

編譯器（`compiler/frontend.py` 中的 `SpiceParser` 類別）執行以下步驟：

1. **預處理**（`preprocess.py`）：
   - 移除註解行（* 開頭）與空行。
   - 處理行延續（+ 開頭的行合併至前一行）。
   - 以 ; 或 $ 截斷行內註解。

- 偵測 .END 指令作為結束標記。

2. **分詞與 AST 建構**（preprocess.py）：
   - 展開括號與逗號為空白分隔的 token。
   - 保留 {...} 內的參數表達式不拆分。
   - 依據首 token 分類為 directive（以 . 開頭）或 element。
3. **參數環境建構**（param_eval.py）：
   - 蒐集所有 .PARAM 指令。
   - 以 Python math 模組函數建立安全的運算命名空間。
   - 儲存於 param_env 字典中。
4. **元件與指令解析**：
   - 遇到 .SUBCKT 時切換作用域至子電路定義內。
   - 遇到 .ENDS 時切回主電路作用域。
   - 元件按首字母前綴路由至對應解析函數。
   - 所有節點名稱統一正規化（GND → 0）。
   - {...} 參數表達式透過 param_env 求值。
5. **子電路展平**（Macro Expansion）：
   - 遞迴展開所有 X 開頭的子電路呼叫。
   - 自動產生前綴（如 X1.R1）避免名稱衝突。
   - 建立內外節點映射表；接地節點（0）永不重新映射。
   - 交叉參照欄位（ctrl_source、element1 等）也同步加上前綴。
6. **驗證**（validator.py）：
   - 確認接地節點存在。
   - 偵測浮接節點（連接數 < 2 的節點）。

編譯器最終輸出一份 JSON 格式的電路描述，包含 elements（元件清單）、analyses（分析指令）、params（參數）、subckts（子電路藍圖）等欄位。

2.4 子電路展平演算法

子電路展平採用遞迴深度優先策略：

```python
def expand(elements, prefix, node_map):
    for el in elements:
        if el["type"] == "subckt_call":
            sub_def = subckts[el["subname"]]
            new_node_map = {}
            for i, pin in enumerate(sub_def["pins"]):
                ext_node = el["pins"][f"p{i}"]
```

```
                    new_node_map[pin] = node_map.get(ext_node, ext_node)
                new_prefix = f"{prefix}{el['name']}."
                expand(sub_def["elements"], new_prefix, new_node_map)
            else:
                new_el = deepcopy(el)
                new_el["name"] = f"{prefix}{new_el['name']}"
                # 重新映射所有非接地節點
                for k, v in new_el["pins"].items():
                    if v != "0":
                        new_el["pins"][k] = node_map.get(v, f"{prefix}{v}")
                flat_elements.append(new_el)
```

Listing 3: 子電路展平核心邏輯

此演算法的時間複雜度為 $O(N_{\text{subckt}} \cdot N_{\text{elements}})$，支援任意深度的巢狀子電路。

第 3 章　修正節點分析法（MNA）

3.1　理論基礎

修正節點分析法（Modified Nodal Analysis, MNA）是 SPICE 模擬器的數學核心。其原理是將電路轉換為聯立線性方程組：

$$\mathbf{A} \cdot \mathbf{x} = \mathbf{b} \tag{1}$$

其中：

- $\mathbf{A}$ 為 $n \times n$ 的係數矩陣（由元件戳印法填入）。
- $\mathbf{x}$ 為未知向量，包含所有非接地節點電壓，以及需要額外變數的元件電流。
- $\mathbf{b}$ 為激勵向量（由獨立源貢獻）。

未知向量的維度為：

$$\dim(\mathbf{x}) = N_{\text{nodes}} + N_{\text{extra}} \tag{2}$$

其中 N_{nodes} 為非接地節點數，N_{extra} 為電壓源、電感器及部分受控源所需的額外電流變數數量。

3.2　元件戳印法（Stamping）

每個元件透過 `stamp()` 方法將自身的電路方程式「戳印」到 $\mathbf{A}$ 矩陣與 $\mathbf{b}$ 向量中。以下逐一推導各元件的戳印規則。

3.2.1　電阻器

電阻器 R 連接節點 n_1 與 n_2，其歐姆定律為：

$$I_{n_1 \to n_2} = G(V_{n_1} - V_{n_2}), \quad G = \frac{1}{R} \tag{3}$$

戳印規則（conductance stamp）：

$$\begin{bmatrix} A[n_1,n_1] & A[n_1,n_2] \\ A[n_2,n_1] & A[n_2,n_2] \end{bmatrix} \mathrel{+}= \begin{bmatrix} +G & -G \\ -G & +G \end{bmatrix} \tag{4}$$

實作中，若 $n_1 = 0$（接地），則跳過該行列的戳印。

3.2.2 電壓源

電壓源 V_s 連接 n_1（正端）與 n_2（負端），引入額外電流變數 i_x（索引 `extra_idx`），其約束為：

$$V_{n_1} - V_{n_2} = V_s \tag{5}$$

同時 i_x 也出現在 KCL 方程中。戳印規則：

$$\begin{aligned} A[n_1,i_x] &\mathrel{+}= +1, \quad A[i_x,n_1] \mathrel{+}= +1 \\ A[n_2,i_x] &\mathrel{+}= -1, \quad A[i_x,n_2] \mathrel{+}= -1 \\ b[i_x] &= V_s \end{aligned} \tag{6}$$

在交流模式下，$b[i_x] = V_{\text{ac}} \cdot e^{j\phi}$（複數相量）。在暫態模式下，$b[i_x]$ 為當下時間點的瞬時波形值。

3.2.3 電流源

電流源 I_s 從 n_1 流向 n_2，不引入額外變數。戳印規則：

$$b[n_1] \mathrel{-}= I_s, \quad b[n_2] \mathrel{+}= I_s \tag{7}$$

3.2.4 電容器

電容器的行為取決於分析模式：

直流模式（.OP）： 電容視為開路，不戳印任何元素。

交流模式（.AC）： 電容的導納為：

$$Y_C = j\omega C, \quad \omega = 2\pi f \tag{8}$$

戳印規則同電阻器，將 G 替換為 Y_C：

$$\begin{bmatrix} A[n_1,n_1] & A[n_1,n_2] \\ A[n_2,n_1] & A[n_2,n_2] \end{bmatrix} \mathrel{+}= \begin{bmatrix} +Y_C & -Y_C \\ -Y_C & +Y_C \end{bmatrix} \tag{9}$$

暫態模式（.TRAN）： 使用後向歐拉法（Backward Euler）離散化：

$$i_C(t) = C\frac{dv}{dt} \approx C\frac{v(t) - v(t-\Delta t)}{\Delta t} \tag{10}$$

定義等效電導與歷史電流：

$$G_{\text{eq}} = \frac{C}{\Delta t}, \quad I_{\text{hist}} = G_{\text{eq}} \cdot v_{\text{prev}} \tag{11}$$

其中 $v_{\text{prev}} = V_{n_1}(t-\Delta t) - V_{n_2}(t-\Delta t)$ 為前一時步的電容跨壓。

戳印規則：

$$\begin{aligned} &\text{矩陣}\mathbf{A} \quad \text{同電阻器，以}G_{\text{eq}}\text{ 取代}G \\ &\text{向量}\mathbf{b} \quad b[n_1] \mathrel{+}= I_{\text{hist}}, \quad b[n_2] \mathrel{-}= I_{\text{hist}} \end{aligned} \tag{12}$$

3.2.5 電感器

電感器引入額外電流變數 i_x，其基本關係為：

$$V_{n_1} - V_{n_2} = L\frac{di}{dt} \tag{13}$$

直流模式： 電感視為短路：

$$\begin{aligned} A[n_1, i_x] &+= 1, \quad A[i_x, n_1] += 1 \\ A[n_2, i_x] &+= -1, \quad A[i_x, n_2] += -1 \\ b[i_x] &= 0 \end{aligned} \tag{14}$$

交流模式： 電感的阻抗為 $Z_L = j\omega L$：

$$A[i_x, i_x] -= j\omega L \tag{15}$$

暫態模式： 後向歐拉法離散化：

$$V_L = L\frac{di}{dt} \approx L\frac{i(t) - i(t - \Delta t)}{\Delta t} \tag{16}$$

定義等效電阻與歷史電壓：

$$R_{\text{eq}} = \frac{L}{\Delta t}, \quad V_{\text{hist}} = -R_{\text{eq}} \cdot i_{\text{prev}} \tag{17}$$

戳印規則：

$$A[i_x, i_x] -= R_{\text{eq}}, \quad b[i_x] += V_{\text{hist}} \tag{18}$$

3.2.6 互感（Mutual Inductance）

互感元件 K 耦合兩個電感器 L_1 與 L_2，互感量為：

$$M = k\sqrt{L_1 L_2}, \quad -1 \le k \le 1 \tag{19}$$

交流模式：

$$Z_M = j\omega M \tag{20}$$

$$A[i_{x1}, i_{x2}] -= Z_M, \quad A[i_{x2}, i_{x1}] -= Z_M \tag{21}$$

暫態模式：

$$R_M = \frac{M}{\Delta t} \tag{22}$$

$$\begin{aligned} A[i_{x1}, i_{x2}] &-= R_M, \quad A[i_{x2}, i_{x1}] -= R_M \\ b[i_{x1}] &+= -R_M \cdot i_{L_2,\text{prev}} \\ b[i_{x2}] &+= -R_M \cdot i_{L_1,\text{prev}} \end{aligned} \tag{23}$$

3.2.7 電壓控制電壓源（VCVS，E 元件）

VCVS 輸出端為 (n_{o+}, n_{o-})，控制端為 (n_{i+}, n_{i-})，增益為 A_v。約束方程：

$$V_{o+} - V_{o-} = A_v(V_{i+} - V_{i-}) \tag{24}$$

引入額外電流變數 i_x，戳印規則：

$$\begin{aligned} &A[n_{o+}, i_x] \mathrel{+}= 1, \quad A[i_x, n_{o+}] \mathrel{+}= 1 \\ &A[n_{o-}, i_x] \mathrel{+}= -1, \quad A[i_x, n_{o-}] \mathrel{+}= -1 \\ &A[i_x, n_{i+}] \mathrel{-}= A_v, \quad A[i_x, n_{i-}] \mathrel{+}= A_v \end{aligned} \tag{25}$$

3.2.8 電壓控制電流源（VCCS，G 元件）

VCCS 不引入額外變數。轉導為 g_m，輸出電流 $I = g_m(V_{i+} - V_{i-})$。

戳印規則：

$$\begin{aligned} &A[n_{o+}, n_{i+}] \mathrel{+}= g_m, \quad A[n_{o+}, n_{i-}] \mathrel{-}= g_m \\ &A[n_{o-}, n_{i+}] \mathrel{-}= g_m, \quad A[n_{o-}, n_{i-}] \mathrel{+}= g_m \end{aligned} \tag{26}$$

3.2.9 電流控制電壓源（CCVS，H 元件）

CCVS 依賴另一電壓源 V_{ctrl} 的電流，轉阻為 R_m。引入額外電流變數 i_x：

$$V_{o+} - V_{o-} = R_m \cdot I_{V_{\text{ctrl}}} \tag{27}$$

戳印規則：

$$\begin{aligned} &A[n_{o+}, i_x] \mathrel{+}= 1, \quad A[i_x, n_{o+}] \mathrel{+}= 1 \\ &A[n_{o-}, i_x] \mathrel{+}= -1, \quad A[i_x, n_{o-}] \mathrel{+}= -1 \\ &A[i_x, i_{\text{ctrl}}] \mathrel{-}= R_m \end{aligned} \tag{28}$$

3.2.10 電流控制電流源（CCCS，F 元件）

CCCS 不引入額外變數。電流增益為 α：

$$I_{\text{out}} = \alpha \cdot I_{V_{\text{ctrl}}} \tag{29}$$

戳印規則：

$$A[n_{o+}, i_{\text{ctrl}}] \mathrel{+}= \alpha, \quad A[n_{o-}, i_{\text{ctrl}}] \mathrel{-}= \alpha \tag{30}$$

3.3 節點管理器

`NodeManager` 類別負責將節點字串名稱映射為整數索引。接地節點 `0` 固定映射為索引 0，不佔用未知數空間。其餘節點依出現順序分配索引 $1, 2, 3, \ldots$，在 MNA 矩陣中的行列索引則為 $0, 1, 2, \ldots$（即節點索引減一）。

第 4 章 時域波形產生器

NextSPICE 支援三種時域波形函數，由共用函數 `eval_source_waveform()` 實作。

4.1 正弦波 SIN

語法：`SIN(VO VA FREQ TD THETA)`

$$v(t) = \begin{cases} V_O & \text{if } t < t_d \\ V_O + V_A \cdot e^{-\theta(t-t_d)} \cdot \sin\bigl(2\pi f(t-t_d)\bigr) & \text{if } t \geq t_d \end{cases} \tag{31}$$

其中：

- V_O：直流偏移量
- V_A：振幅
- f：頻率 (Hz)
- t_d：延遲時間
- θ：阻尼係數（$\theta = 0$ 時為無阻尼正弦波）

4.2 脈衝波 PULSE

語法：`PULSE(V1 V2 TD TR TF PW PER)`

定義 $t_c = (t - t_d) \bmod T_{\text{per}}$，則：

$$v(t) = \begin{cases} V_1 & \text{if } t < t_d \\ V_1 + (V_2 - V_1)\dfrac{t_c}{t_r} & \text{if } 0 \leq t_c < t_r \\ V_2 & \text{if } t_r \leq t_c < t_r + t_{pw} \\ V_2 - (V_2 - V_1)\dfrac{t_c - t_r - t_{pw}}{t_f} & \text{if } t_r + t_{pw} \leq t_c < t_r + t_{pw} + t_f \\ V_1 & \text{otherwise} \end{cases} \tag{32}$$

其中 V_1：初始值，V_2：脈衝值，t_r：上升時間，t_f：下降時間，t_{pw}：脈衝寬度，T_{per}：週期。

4.3 分段線性波 PWL

語法：`PWL(T1 V1 T2 V2 T3 V3 ...)`

給定 n 個時間–數值對 $\{(t_i, v_i)\}_{i=1}^{n}$，波形值以線性內插計算：

$$v(t) = \begin{cases} v_1 & \text{if } t \leq t_1 \\ v_n & \text{if } t \geq t_n \\ v_i + (v_{i+1} - v_i)\dfrac{t - t_i}{t_{i+1} - t_i} & \text{if } t_i \leq t \leq t_{i+1} \end{cases} \tag{33}$$

第 5 章　分析演算法

5.1 直流工作點分析（.OP）

直流工作點分析求解穩態下的所有節點電壓與支路電流。

演算法步驟：

1. 呼叫 `_prepare_mna_structure()` 計算 MNA 維度：統計非接地節點數與額外變數數。
2. 建立 SciPy 稀疏 LIL 矩陣 $\mathbf{A}$（`lil_matrix`）與零向量 $\mathbf{b}$。
3. 遍歷所有元件，呼叫 `stamp()` 填入矩陣。
4. 將 LIL 矩陣壓縮為 CSR 格式（Compressed Sparse Row）。
5. 呼叫 `scipy.sparse.linalg.spsolve()` 求解 $\mathbf{x} = \mathbf{A}^{-1}\mathbf{b}$。
6. 計算殘差 $r = \|\mathbf{Ax} - \mathbf{b}\|_\infty$ 以驗證解的精度。

特殊處理：

- 電容器在直流模式下不戳印（視為開路）。
- 電感器在直流模式下戳印為短路（$b[i_x] = 0$）。

5.2 交流小訊號分析（.AC）

交流分析在複數域中進行頻率掃描，求解各節點的頻率響應。

頻率點產生：

- `DEC`：以十倍頻為單位的對數分布，$f_k = 10^{\log_{10}(f_{\text{start}}) + k \cdot \Delta}$。
- `OCT`：以八度音為單位的對數分布。
- `LIN`：等距線性分布。

每個頻率點的計算：

1. 建立複數稀疏矩陣（`dtype=np.complex128`）。
2. 各元件以 $\omega = 2\pi f$ 計算阻抗：
   - 電阻：$Z_R = R$（不變）
   - 電容：$Y_C = j\omega C$
   - 電感：$Z_L = j\omega L$
3. 求解複數方程組 $\mathbf{A}_{\text{ac}}\mathbf{x}_{\text{ac}} = \mathbf{b}_{\text{ac}}$。
4. 提取結果：

$$|\text{Mag}|_{\text{dB}} = 20 \log_{10} |V_{\text{node}}| \tag{34}$$

$$\phi = \arg(V_{\text{node}}) \quad \text{（度）} \tag{35}$$

5.3 直流掃描分析（.DC）

直流掃描對某一獨立源的直流值進行參數掃描。

演算法：

1. 找到目標源元件。
2. 產生掃描點序列 $\{v_k\}_{k=0}^{N}$，其中 $v_k = v_{\text{start}} + k \cdot \Delta v$。
3. 對每個掃描點 v_k：
   (a) 修改目標源的 `dc_value` 為 v_k。
   (b) 執行一次完整的 `.OP` 求解。
   (c) 記錄所有節點電壓。
4. 復原目標源的原始值。

5.4 暫態分析（.TRAN）

暫態分析以後向歐拉法（Backward Euler Method，隱式一階方法）在時域中逐步推進模擬。

5.4.1 後向歐拉法原理

對於一般的常微分方程 $\dot{y} = f(t, y)$，後向歐拉法的遞推公式為：

$$y_{n+1} = y_n + \Delta t \cdot f(t_{n+1}, y_{n+1}) \tag{36}$$

此方法的關鍵特性為**無條件穩定**（unconditionally stable），意味著無論時步 Δt 多大都不會發散。代價是只有一階精度，即截斷誤差為 $O(\Delta t)$。

5.4.2 伴隨電路模型（Companion Model）

後向歐拉法將動態元件（電容、電感）在每個時步轉化為等效的靜態電路模型：

電容器的伴隨模型： 由 $i_C = C\, dv/dt$ 離散化後，電容器在每個時步等效為一個並聯的電導 G_{eq} 與電流源 I_{hist}：

$$\boxed{G_{\text{eq}} = \frac{C}{\Delta t}, \quad I_{\text{hist}} = \frac{C}{\Delta t} \cdot v_{\text{prev}}} \tag{37}$$

電感器的伴隨模型： 由 $v_L = L\, di/dt$ 離散化後，電感器在每個時步等效為一個串聯的電阻 R_{eq} 與電壓源 V_{hist}：

$$\boxed{R_{\text{eq}} = \frac{L}{\Delta t}, \quad V_{\text{hist}} = -\frac{L}{\Delta t} \cdot i_{\text{prev}}} \tag{38}$$

5.4.3 暫態分析流程

1. **初始條件**：在 $t = 0$ 時，將所有具暫態波形的源設為其 $t = 0$ 的瞬時值，然後執行一次 `.OP` 求解，得到初始解向量 $\mathbf{x}_0$。
2. **歷史更新**：從 $\mathbf{x}_0$ 提取所有電容跨壓和電感電流，儲存為 v_{prev} 與 i_{prev}。
3. **時間步進迴圈**：對每個時步 $t_k = k \cdot \Delta t$：
   (a) 建立全新的稀疏 LIL 矩陣。
   (b) 評估所有時域波形在 t_k 的瞬時值。
   (c) 各元件以暫態模式（含伴隨模型）戳印矩陣。
   (d) 壓縮為 CSR 格式並求解。
   (e) 更新所有動態元件的歷史狀態。

歷史更新公式： 電容器：

$$v_{\text{prev}} \leftarrow x[n_1 - 1] - x[n_2 - 1] \tag{39}$$

電感器：

$$i_{\text{prev}} \leftarrow x[i_x] \tag{40}$$

5.5 靈敏度分析（.SENS）

靈敏度分析使用**中央差分法**（Central Difference Method）計算各元件參數對直流增益的影響。

5.5.1 直流增益定義

$$G_{\text{base}} = \frac{V_{\text{out}}}{V_{\text{in}}} \tag{41}$$

其中 V_{out} 為指定輸出節點電壓，V_{in} 為指定輸入源的直流值。

5.5.2 中央差分靈敏度

對每個被測元件的參數 P：

1. 動態計算微擾量：
$$\Delta P = \max\big(|P| \cdot \epsilon_{\text{rel}},\ \epsilon_{\text{min}}\big) \tag{42}$$
   預設 $\epsilon_{\text{rel}} = 10^{-5}$，$\epsilon_{\text{min}} = 10^{-12}$。
2. 正向微擾：將參數設為 $P + \Delta P$，求解 `.OP`，計算增益 G^+。
3. 反向微擾：將參數設為 $P - \Delta P$，求解 `.OP`，計算增益 G^-。
4. 立即復原參數原始值。
5. 絕對靈敏度：
$$S_{\text{abs}} = \frac{G^+ - G^-}{2\,\Delta P} \tag{43}$$

6. 正規化靈敏度：

$$S_{\text{norm}} = \frac{P}{G_{\text{base}}} \cdot S_{\text{abs}} \tag{44}$$

正規化靈敏度的物理意義為：當參數 P 變化 1% 時，增益 G 大約變化 $S_{\text{norm}}\%$。

5.6 參數掃描（.STEP）

`.STEP` 指令在最外層包裹一個迴圈，對指定元件的參數值進行掃描。每個掃描步驟都會重新建立 `Simulator` 實例並執行所有已定義的分析。

$$P_k = P_{\text{start}} + k \cdot \Delta P, \quad k = 0, 1, \ldots, \left\lfloor \frac{P_{\text{stop}} - P_{\text{start}}}{\Delta P} \right\rfloor \tag{45}$$

第 6 章 量測引擎

6.1 .MEASURE TRAN 指令

量測引擎（`measure.py`）實作了 `.MEASURE TRAN` 語法，用於從暫態模擬結果中提取特定的時間特徵。

6.1.1 TRIG...TARG 雙點測量語法

```
.MEASURE TRAN delay TRIG V(IN) VAL=2.5 RISE=1
+                   TARG V(OUT) VAL=2.5 RISE=1
```

此指令表示：量測從 `V(IN)` 第一次上升穿越 2.5V 到 `V(OUT)` 第一次上升穿越 2.5V 之間的時間差。

6.1.2 穿越點偵測演算法

對於波形 $y(t)$ 與目標值 v_{th}，系統在相鄰取樣點之間偵測穿越事件：

- **上升穿越**（RISE）：$y[i] < v_{\text{th}} \leq y[i+1]$
- **下降穿越**（FALL）：$y[i] > v_{\text{th}} \geq y[i+1]$
- **任意穿越**（CROSS）：上升或下降皆計入

穿越時間以線性內插計算：

$$t_{\text{cross}} = t_i + \frac{v_{\text{th}} - y_i}{y_{i+1} - y_i} \cdot (t_{i+1} - t_i) \tag{46}$$

最終量測值為兩個穿越時間的差：

$$\text{delay} = t_{\text{TARG}} - t_{\text{TRIG}} \tag{47}$$

第 7 章　稀疏矩陣求解策略

7.1　為何使用稀疏矩陣

電路的 MNA 矩陣具有高度稀疏性：一個有 n 個節點的電路，其 MNA 矩陣維度為 $n \times n$，但非零元素數量通常僅為 $O(n)$ 等級（因為每個元件最多影響 4 個矩陣位置）。

矩陣類型	建構時間	求解時間	記憶體
稠密矩陣	$O(n^2)$	$O(n^3)$	$O(n^2)$
稀疏矩陣	$O(n_z)$	$O(n_z^2)$（最壞）	$O(n_z)$

其中 n_z 為非零元素數。對於大規模電路（$n > 100$），稀疏矩陣可帶來約 1000 倍的速度提升。

7.2　SciPy 稀疏矩陣工作流程

NextSPICE 的稀疏矩陣策略分為三個階段：

1. **LIL 格式建構**（List of Lists）：

   以 `scipy.sparse.lil_matrix` 建立矩陣。LIL 格式最適合動態逐元素插入，因為它以鏈結串列儲存每一列的非零元素。

2. **CSR 格式壓縮**（Compressed Sparse Row）：

   戳印完成後，呼叫 `.tocsr()` 將 LIL 矩陣轉換為 CSR 格式。CSR 以三個一維陣列儲存非零值、列指標與列偏移量，是最高效的矩陣向量運算格式。

3. **稀疏直接求解**：

   呼叫 `scipy.sparse.linalg.spsolve()` 進行 LU 分解求解。底層使用 SuperLU 或 UMFPACK 演算法。

第 8 章　電路建構器（Two-Pass 建構法）

`Circuit` 類別（`runtime/circuit.py`）負責將編譯器輸出的 JSON 藍圖實體化為 Python 元件物件。為了正確處理元件間的交叉參照關係，建構過程分為兩個 Pass：

8.1　Pass 1：獨立元件建構

在第一輪中建構所有不依賴其他元件的基礎元件：

- 電阻器（R）、電容器（C）、電感器（L）
- 電壓源（V）、電流源（I）
- 電壓控制源（E, G）

每個元件建構後立即註冊到 `_element_by_name` 字典，以支援 $O(1)$ 的名稱查找。

8.2 Pass 2：交叉參照解析

在第二輪中處理需要引用其他元件的延遲元件：

- **互感（K）**：需引用兩個電感器物件 L_1 與 L_2。
- **CCVS（H）**：需引用控制電壓源的名稱以取得其電流索引。
- **CCCS（F）**：同 H 元件。

若引用的目標元件不存在，系統會拋出明確的錯誤訊息。

第 9 章 分析排程總管

`SimulationRunner`（`runtime/runner.py`）是最高層的分析協調器，負責：

1. 解析 `.STEP` 配置，產生外層參數掃描迴圈。
2. 在每個掃描步驟中，建立 `Simulator` 實例。
3. 依序執行所有已定義的分析（`.OP`、`.TRAN`、`.AC`、`.DC`、`.SENS`）。
4. 將結果打包為統一的 JSON 格式。

9.1 結果輸出格式

Runner 產出的 JSON 結構包含：

```
{
  "status": "success",
  "logs": ["[INFO] ...", ...],
  "op_results": {
    "V(IN)": 10.0,
    "V(OUT)": 5.0,
    "I(V1)": -0.005,
    "P(R1)": 0.025
  },
  "plots": [
    {
      "name": "V(OUT)",
      "x": [0.0, 0.001, ...],
      "y": [5.0, 5.1, ...],
      "type": "solid"
    }
  ],
  "layout": {
    "title": "Transient Response",
    "xaxis": "Time (s)",
    "yaxis": "Voltage (V)"
  }
}
```

Listing 4: 輸出結構

9.2 功率計算

Runner 會自動為以下元件計算功率消耗：

- 電阻器：$P_R = \dfrac{(V_{n_1} - V_{n_2})^2}{R}$
- 電壓源：$P_V = V_{\text{dc}} \cdot I_V$

第 10 章 支援的元件總覽

前綴	類型	端子	額外變數	說明
R	被動	2	0	電阻器（值 > 0）
C	被動	2	0	電容器（值 ≥ 0）
L	被動	2	1	電感器（值 ≥ 0）
K	被動	—	0	互感（$\lvert k\rvert \leq 1$）
V	獨立源	2	1	電壓源（DC/AC/TRAN）
I	獨立源	2	0	電流源（DC/AC/TRAN）
E	受控源	4	1	VCVS（電壓控制電壓源）
G	受控源	4	0	VCCS（電壓控制電流源）
H	受控源	2+ref	1	CCVS（電流控制電壓源）
F	受控源	2+ref	0	CCCS（電流控制電流源）
X	階層	可變	—	子電路呼叫
D	非線性	2	—	二極體（僅解析，未求解）

第 11 章 完整模擬範例

11.1 範例一：RC 低通濾波器頻率響應

```
RC Low Pass Filter
V1 IN 0 AC 1 0
R1 IN OUT 1k
C1 OUT 0 100n
.AC DEC 100 10 1MEG
.END
```

Listing 5: RC 低通濾波器

理論分析： 此電路的轉移函數為：

$$H(j\omega) = \frac{V_{\text{out}}}{V_{\text{in}}} = \frac{1}{1 + j\omega RC} \tag{48}$$

截止頻率：

$$f_c = \frac{1}{2\pi RC} = \frac{1}{2\pi \times 1000 \times 100 \times 10^{-9}} \approx 1.59\,\text{kHz} \tag{49}$$

11.2 範例二：RLC 暫態響應

```
RLC Transient
V1 IN 0 PULSE(0 5 1m 1u 1u 4m 10m)
L1 IN N1 10m
C1 N1 0 1u
R1 N1 0 10
.TRAN 10u 10m
.END
```

Listing 6: RLC 暫態響應

理論分析： 此 RLC 串並聯電路的自然頻率與阻尼比為：

$$\omega_0 = \frac{1}{\sqrt{LC}} = \frac{1}{\sqrt{10^{-2} \times 10^{-6}}} \approx 10{,}000 \text{ rad/s} \tag{50}$$

$$\zeta = \frac{1}{2R}\sqrt{\frac{L}{C}} = \frac{1}{2 \times 10}\sqrt{\frac{10^{-2}}{10^{-6}}} = 5 \tag{51}$$

由於 $\zeta > 1$，此電路為過阻尼（overdamped），不會出現振盪。

11.3 範例三：變壓器互感模型

```
Transformer
V1 P_IN 0 SIN(0 10 1k)
R1 P_IN P1 1
L1 P1 0 10m
L2 S1 0 40m
K1 L1 L2 0.99
R2 S1 0 100
.TRAN 10u 5m
.END
```

Listing 7: 變壓器模型

理論分析： 匝比 n 由電感比推導：

$$n = \sqrt{\frac{L_2}{L_1}} = \sqrt{\frac{40\text{m}}{10\text{m}}} = 2 \tag{52}$$

理想情況下（$k = 1$），次級開路電壓為初級電壓的 n 倍。

11.4 範例四：含參數的靈敏度分析

```
Sensitivity Test
V1 IN 0 DC 10
R1 IN MID 1k
R2 MID OUT 2k
R3 OUT 0 3k
```

```
.SENS V(OUT) V1
.END
```

Listing 8: 靈敏度分析

手動驗證： 使用分壓器公式，V_{OUT} 對 R_3 的靈敏度可由解析方法驗證，中央差分結果應與之吻合。

第 12 章　端到端資料流程圖

以下為 NextSPICE 從網表輸入到結果輸出的完整資料流程：

1. **輸入**：使用者提交 SPICE 網表文字。
2. **編譯階段**：
   - 2.1 預處理 → 移除註解、合併延續行。
   - 2.2 分詞 → 產生 token 序列。
   - 2.3 AST 建構 → 分類為 element / directive。
   - 2.4 參數收集 → 建構 `param_env`。
   - 2.5 元件解析 → 產生 JSON 元件物件。
   - 2.6 指令解析 → 產生分析配置。
   - 2.7 子電路展平 → 遞迴展開所有 X 呼叫。
   - 2.8 驗證 → 檢查接地與節點連通性。
3. **建構階段**：
   - 3.1 Pass 1 → 建立獨立元件物件。
   - 3.2 Pass 2 → 解析交叉參照（K, H, F）。
   - 3.3 節點映射 → 字串 → 整數索引。
4. **模擬階段**（每個.STEP 步驟）：
   - 4.1 建立 Simulator 實例。
   - 4.2 計算 MNA 維度，分配額外變數索引。
   - 4.3 依分析類型執行：
     - `.OP`：建構稀疏矩陣 → 戳印 → CSR 壓縮 → 求解。
     - `.AC`：頻率掃描 ×（複數矩陣建構 → 戳印 → 求解）。
     - `.DC`：參數掃描 × `.OP`。
     - `.TRAN`：初始 `.OP` → 時步迴圈 ×（伴隨模型戳印 → 求解 → 歷史更新）。
     - `.SENS`：基準增益 → 元件微擾迴圈 ×（$\pm\Delta P$ 各一次 `.OP`）。
5. **量測階段**：執行 `.MEASURE` 指令，線性內插取值。
6. **輸出**：JSON 結構，包含日誌、工作點結果、波形圖資料與佈局資訊。

第 13 章　已知限制與未來展望

13.1　目前限制

1. **非線性元件**：二極體（D）已可解析但尚未在求解器中實作 Newton-Raphson 迭代。
2. **電晶體**：尚未支援 BJT（Q）與 MOSFET（M）模型。
3. **自適應時步**：暫態分析使用固定時步，尚未實作基於截斷誤差的自適應步長控制。
4. **收斂控制**：缺乏限流（limiting）與阻尼（damping）機制來輔助非線性收斂。
5. **行為源**：尚不支援任意數學表達式的 B 元件。
6. **高階積分法**：僅實作一階後向歐拉法，尚未支援梯形法（Trapezoidal）或 Gear 法。

13.2　效能特性

演算法	複雜度	狀態
MNA 元件戳印	$O(1)$ /元件	完成
OP 稀疏求解	$O(n_z^2)$	完成
AC 頻率掃描	$O(N_f \times n_z^2)$	完成
DC 參數掃描	$O(N_s \times n_z^2)$	完成
暫態時域積分	$O(N_t \times n_z^2)$	完成
靈敏度微擾	$O(N_p \times n_z^2)$	完成
子電路展平	$O(N_X \times N_e)$	完成

附錄 A：數學符號表

符號	說明
$\mathbf{A}$	MNA 係數矩陣
$\mathbf{x}$	未知向量（節點電壓＋支路電流）
$\mathbf{b}$	激勵向量
G	電導 $(1/R)$
Y_C	電容導納 $(j\omega C)$
Z_L	電感阻抗 $(j\omega L)$
G_{eq}	電容伴隨電導 $(C/\Delta t)$
R_{eq}	電感伴隨電阻 $(L/\Delta t)$
I_{hist}	電容歷史電流
V_{hist}	電感歷史電壓
M	互感量 $(k\sqrt{L_1 L_2})$
ω	角頻率 $(2\pi f)$
Δt	時步大小

S_{abs}	絕對靈敏度
S_{norm}	正規化靈敏度
n_z	稀疏矩陣非零元素數

附錄 B：SPICE 網表快速參考

```
* === 被動元件 ===
Rname N+ N- value
Cname N+ N- value
Lname N+ N- value
Kname Lname1 Lname2 coupling

* === 獨立源 ===
Vname N+ N- [DC value] [AC mag [phase]]
+                      [SIN(VO VA FREQ TD THETA)]
+                      [PULSE(V1 V2 TD TR TF PW PER)]
+                      [PWL(T1 V1 T2 V2 ...)]
Iname N+ N- [DC value] [AC mag [phase]]

* === 受控源 ===
Ename N+ N- NC+ NC- gain        ; VCVS
Gname N+ N- NC+ NC- gm          ; VCCS
Hname N+ N- Vctrl rm            ; CCVS
Fname N+ N- Vctrl gain          ; CCCS

* === 子電路 ===
.SUBCKT name pin1 pin2 ...
  ... elements ...
.ENDS [name]
Xname ext1 ext2 ... subckt_name

* === 分析指令 ===
.OP
.AC DEC|LIN|OCT points fstart fstop
.DC source start stop step
.TRAN tstep tstop
.SENS V(node) [source]
.STEP PARAM name start stop step

* === 其他 ===
.PARAM name=value
.MEASURE TRAN name TRIG ... TARG ...
.END
```

Listing 9: 完整語法快速參考